The kernel and its modules

Drishti · docs/architecture/KERNEL_AND_MODULES.md · Copyright © 2026 Ashutosh Sinha. All rights reserved. Proprietary and confidential.

Status: design, not built. Written for 2.9.11.i (*architecture optimisation*), on the product owner's request of 2026-11-08: Drishti becomes a domain-free **kernel** for BI and aggregation; capabilities arrive as separately built, signed **modules** loaded from folders; connectors stay the kernel's; a **distribution file** white-labels the product. Banking is the first module and the worked example throughout. Surveyed on `wip/train 89f010df5`. A printable copy with the diagrams and mock-ups: KERNEL_AND_MODULES.pdf.

This is the one home for the kernel, the module package, loading, data binding, distributions and the SDK. It builds on what is already built and does not repeat it: the access chain (licence → module → lens → function → permission) and the registry are ACCESS_MODEL.md; the boundary survey, the dependency rules and the split as built are DOMAIN_MODULES.md (section 12 says what moves from it into this document); the JavaScript libraries are ELEMENTS.md; the lakes and connectors are DATA_ARCHITECTURE.md; the lenses are LENSES.md.

Worked examples (files, not product code) are in `docs/architecture/kernel-and-modules/examples/` (the banking manifest: module.yaml); the screens' mock-ups in `docs/architecture/kernel-and-modules/mockups/` (section 13).

Contents

1. Goals, non-goals and principles
2. The kernel
3. The module package (`.dmod`)
4. Loading
5. Data needs and connector binding
6. The distribution file (white label)
7. Lifecycle and tooling
8. The Module SDK
9. Security
10. Migration, with banking as the proving ground
11. Open questions for the owner
12. What moves from DOMAIN_MODULES.md
13. Mock-ups

---

1. Goals, non-goals and principles

1.1 Goals

#	Goal	Proof
G1	**A domain-free kernel**: general BI and aggregation (packs runtime, Sutras, Rachana, Pivot, BI, tables and models, lakes, connectors, search, Explain, approvals, security, cluster, Elements core) that serves a bank, a hospital or a genome lab unchanged	the kernel boots and serves BI, Pivot and its own reference data with **no module installed**; `tools/domain_strings_allowed.yaml` is empty
G2	**Modules built and shipped separately**, like Linux kernel modules: one signed file (`.dmod`) per module, compiled against the kernel API only, dropped into a configured folder	the banking module is built in its own build against the SDK jars and passes its suite against the kernel alone
G3	**A module carries everything its capability needs**: Java compute, services and API; Python compute; packs (kinds, contracts, Sutras, samples); console routes, templates, the domain Elements library, CSS on the theme tokens; lenses, functions, Home cards; feeds; Help, guides, tutorials, glossary; settings schema; licence feature	removing `banking.dmod` leaves no banking word, page, file, Help page, engine or API anywhere (ACCESS_MODEL.md section 6, extended to "not installed")
G4	**The kernel controls connectors.** A module declares the *data* it needs (kinds and contracts); the site binds its configured connectors and lake tables to them	no connector name, plugin id, host or credential in any module file (a packager check); Admin → Data bindings shows every need and what serves it
G5	**White label by configuration**: one distribution file picks modules, brand, theme, lenses, Home and docs	`drishti.finance` and a sample `drishti.genomics` boot from configuration only, from the same kernel build
G6	**VaR exactly equal** through every phase	the 2.9.11.c VaR check, bit for bit, at the 509 and 250 windows

1.2 Non-goals

- No public module marketplace or online repository in 2.9.11.i (a site's module repository is phase P5).
- No third-party connector plugins inside modules, ever (G4). Connector plugins remain kernel extensions (section 2.4).
- No OSGi or JPMS layer: one class loader per module with an explicit export list is enough and keeps Spring and the build simple.
- No second way: banking does not keep an "in the jar" path once it is a `.dmod` (no dual machinery: the development fat jar is a build of the kernel plus modules, not a different loading path; section 4.10).
- No hot load of code in P2 (packs and Sutras are already hot; code modules restart). Hot code load is P5.

1.3 Principles

1. **The kernel knows no domain.** No banking word, engine id, kind, path or package in kernel code, templates, scripts or styles. The rule-4 check of DOMAIN_MODULES.md section 4 becomes a gate with an empty allow list.
2. **One way. Banking ships as a module like any other.** No privileged module, no "built-in" path. Economy and capital markets are content-only modules packaged the same way. The kernel's own reference data (calendar, GLEIF) stays kernel content.
3. **Modules reach the kernel only through the kernel API** (section 2.2): versioned SPIs and services, the Rachana grammar, the data-contract format, the Elements core API, the REST namespace given to the module, the event bus. Everything else of the kernel is invisible to a module's class loader.

4. **Modules never depend on each other's internals.** `requires:` names a module and a version range; a module may use another's *published kinds and functions* (data), never its classes.
5. **Standard libraries, standard formats.** The package is a zip with a JAR-style signature; YAML manifests; JSON Schema for settings; ES modules and `.d.ts` for JavaScript; semver for versions; data in JSONL and Delta/Iceberg via their official libraries.
6. **Secure by default.** Unsigned or untrusted modules are refused (a development key exists for the SDK harness only); licences decide what loads; Python runs in the kernel's sandbox; module assets live under a strict CSP; **secrets never enter a module**.
7. **Elements, never bespoke UI.** A module's screens are built from Drishti Elements and its own domain library (drishti-risk for banking), on the kernel's theme tokens (ELEMENTS.md).
8. **Refused means absent, with the reason.** A module that fails a check is not loaded at all; Health and Admin → Modules say why in words.

1.4 The Linux analogy (the owner's frame)

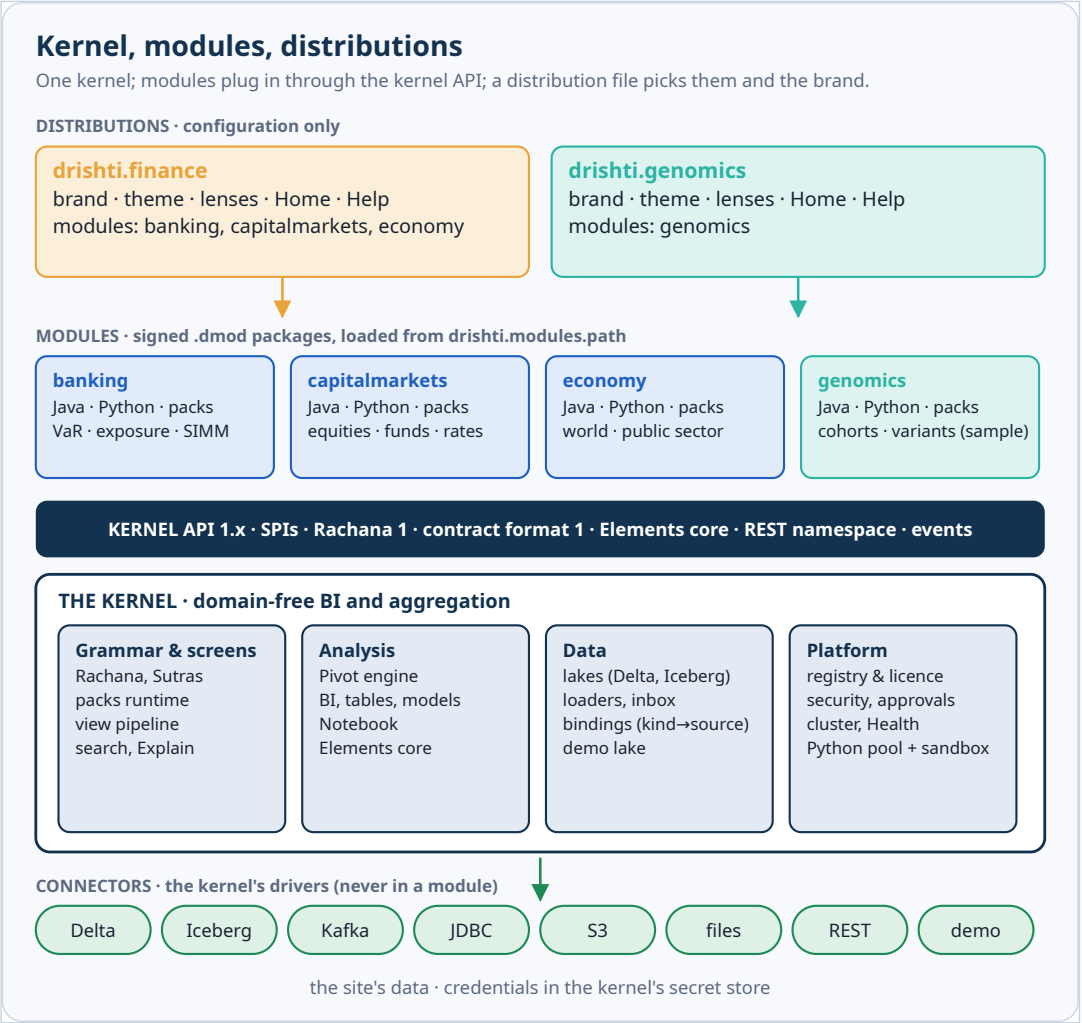
The Linux-module analogy

The owner's frame: modules are compiled and packaged apart, and the kernel runs them.

Linux	Drishti
`kernel`	Drishti kernel (server, console, Elements core, connectors)
`.ko module`	.dmod module package
`module ABI · vermagic`	kernel API version · manifest's kernel: range
`modinfo`	module.yaml · module info
`depmod · modprobe`	requires: with version ranges, resolved first
`module signing · Secure Boot`	signed .dmod · trusted publishers · licence
`insmod · rmmod · lsmod`	module install · remove · list · Admin → Modules
`taint`	Health: TAINTED (dev-signed, override …)
`device drivers`	connectors: the kernel's, bound to kinds
`distribution (Debian …)`	distribution file (drishti.finance …)

Linux	Drishti
kernel	the Drishti kernel (server + console core + Elements core + connector plugins)
`.ko` module	`.dmod` module package
module ABI, `vermagic`	the versioned kernel API and the manifest's `kernel:` range
`modinfo`	`module.yaml` (and `drishti.py module info`)
`depmod` / `modprobe` dependency resolution	`requires:` with version ranges, resolved before loading
module signing, Secure Boot	signed `.dmod`, trusted publisher keys, licence through the registry
`insmod` / `rmmod` / `lsmod`	`drishti.py module install | remove | list`, Admin → Modules
taint	Health "unsupported module loaded" (development-signed, out-of-range, or overridden)
device drivers talk to hardware via the kernel's bus	modules read data via kernel **bindings**; connectors are the kernel's drivers
distribution (Debian, Fedora)	distribution file (`drishti.finance`, `drishti.genomics`)

2. The kernel



2.1 What is in it

Area	Kernel (Maven modules / folders)
Foundations	`drishti-common`, `drishti-api` (data SPIs), `drishti-registry` (access registry, licence), `drishti-identity`
Grammar and screens	`drishti-rachana` (Rachana, Sutras), `drishti-engine` (view pipeline, inference, search, phrase, view augmenters), `drishti-packs` (pack loader and pack runtime)
Analysis	`drishti-bicompute` (pivot engine, Rachana functions, Explain, what-if), `drishti-tables` (virtual tables, models), `drishti-graph`, `drishti-inference`
Data	`drishti-lakeio`, `drishti-lakeconfig`, `drishti-lakeload`, `drishti-deltalake`, `drishti-iceberg`, `drishti-memlake`, `drishti-diskcache`, `drishti-dataload` (the inbox), `drishti-flight`, `drishti-messaging`
Connectors	every `plugins/drishti-plugin-*` (delta, iceberg, iceberg-external, file, s3, jdbc, duckdb, kafka, rabbitmq, activemq, redis, mongodb, cassandra, aerospike, rest, feeds, news, demo, yfinance); the connector files, credentials and **bindings** (section 5)
Compute	`drishti-compute` (generic: vector store, many-set sums, sharding, the Python pool and sandbox, the scheduler), `drishti-native` (BLAS/LAPACK with Java fallbacks)
Platform	`drishti-server` (REST, security, approvals, collaboration, alerts, cluster `drishti-net`, Health, audit, Admin), `drishti-ai` (LLM providers, MCP), `drishti-app` (the runnable kernel jar)
Console	`drishti-console` core: shell, command line, Home, Pivot, BI, Build, Notebook, Admin, Help centre, search, embed, phone; `core/plugins.py` hooks
JavaScript	Elements libraries 1–7 and the generic additions: drishti-core, -inputs, -grid, -chart (+ maps), -data, -insight, -renderer, -graph, -tree, -designer
Content	the core pack (calendar and generic entities), GLEIF reference data, the compute-demo pack, the kernel's Help (generic guides and tutorials), the CIO and oversight lenses, the default theme

Not in it: `drishti-banking`, `drishti-compute-banking`, `drishti-console/plugins/{banking,economy}`, the drishti-risk library, the packs of `config/packs/{banking,economy,capitalmarkets}`, their lenses, lakes, BI content, feeds, guides and tutorials. Each of those is a module (section 10.1 lists today's leaks).

2.2 The kernel API

The kernel API is a **published, versioned contract**: what a module may compile against and rely on. It is one version number for the whole surface, `kernelApi: MAJOR.MINOR`, separate from the product version (2.9.11.i ships kernel API **1.0**).

Surface	Form	Where
Java SPIs and services	`drishti-kernel-api-<v>.jar` (interfaces, records, annotations only; no implementation)	section 2.3
Python compute API	`drishti_kernel` package (the compute SDK's item, frame and result types; the sandbox's rules)	section 4.6
Rachana and Sutras	grammar version `rachana: 1` (already in every Sutra)	RACHANA_REFERENCE.md
Data contracts	contract format `contract-format: 1` (the `*.contract.yaml` shape the packs use today)	section 5.2
Elements core	`@drishti/elements-core` etc.: ES modules + `.d.ts`, the theme tokens	ELEMENTS.md
Console plugin API	`plugin.yaml` format 1 and the hooks of `core/plugins.py` (command, retired, type_ahead, hints, preset, landing_rule, screens)	section 4.7
REST	the namespace a module is given (`/api/v1/<module api>/…`) and the kernel's public endpoints	section 2.5
Events	the kernel event catalogue	section 2.6
Manifest	`module.yaml` format 1	section 3.3

Versioning rules.

- **Semver on the kernel API**, not on the product: a MINOR adds (a new SPI, a new optional method with a default, a new event); a MAJOR removes or changes. The product version and the kernel API version are both printed on About, Health and `drishti.py module list`.
- **Deprecation over one release** (the owner's rule): a deprecated element keeps working for one product release, logged once per start and listed on Admin → Modules against each module that still uses it ("banking 3.0.0 uses `HealthFigures.contribute(Map)`, deprecated, gone in the next release"); the next release removes it and bumps MAJOR.
- **Compatibility range**: the kernel declares `kernelApi: 1.4` and `serves: [1.3, 1.4]` (the current minor and the one before it within the major). A module declares `kernel: ">=1.3 <2.0"`. A module is loadable when its range contains the kernel's `kernelApi` and its lowest bound is not older than `serves`' lowest (the `vermagic` check, section 4.3).
- **The SDK jar is the contract**: anything not in `drishti-kernel-api` is not API, even if public in `drishti-server`. An architecture test fails a module class that imports anything else of `com.ash.drishti..` (section 4.4).
- **API diffs are generated** each release (`japicmp` on the API jar, a `.d.ts` diff, a manifest-schema diff) and a MAJOR change without a MAJOR bump fails the gate.

2.3 The SPIs

Existing SPIs (built for the domain split; the banking module fills them today). Each moves into `drishti-kernel-api` under `com.ash.drishti.kernel.api.*`, keeping its shape; banking's use is in brackets.

SPI (today's home)	What a module contributes	Banking
`ComputeEngine` (`drishti-api` compute)	a compute engine with its switch, words and health	VaR, exposure, FRTB-SA, SIMM, sensitivities, MR cube
`PivotProvider` (`drishti-bicompute` spi)	measures the pivot engine computes per cell (non-additive totals)	VaR, ES, PVaR by any grouping
`PivotSourceProvider` (`server.pivot`)	a source the pivot asks before the tabular provider	the VaR source of the MR lens
`FunctionProvider` (`drishti-rachana` el)	Rachana functions (`var()`, `pvar()`, `es()`)	VaR functions in BI and Sutras
`ViewAugmenter` (`drishti-engine`)	fields added to a kind's view as the page opens	`VAR_ENGINE.*` on a position, trade, book
`Explainer` (`server.explain`)	the Explain drawer's formula, inputs and lineage for its numbers	VaR, component VaR, EPE, FRTB charge
`CacheGroup` / `CacheReport.Owner` (`server.caches`)	caches the kernel reports and clears in order	VaR and exposure date caches
`ApprovalSource` (`server.approvals`)	items in the Approvals inbox	regulatory VaR sign-off, scope changes
`ShardedMeasure` (`server.shard`)	a measure computed across shards	VaR by hierarchy at scale
`FlightForm` (`server.flight`)	an Arrow Flight descriptor it serves	P&L vectors out
`CapacityFigures` (`server.capacity`)	its rows in the capacity planner	VaR memory and time per date
`HealthFigures` (`server.api`)	figures in Health	hierarchies skipped
`OfficialReaders.OfficialResults` (`drishti-packs`)	published results lakes it reads	`var-official`, `ccr-official`
`ApiAlias` (`server.modules`)	old paths answered as new for one release	`/api/v1/var` → `/api/v1/risk/var`
`SourcePlugin` (`drishti-api`)	**kernel only from P1** (section 5.7): banking's `VarPositionsPlugin` and `VarPvStreamPlugin` become `KindProvider`s	—
`StorageProvider` (`drishti-lakeio`), `ProviderFactory` (`drishti-ai`)	**kernel only** (storage back-ends, LLM providers)	—

New SPIs (P1 unless noted):

SPI	Purpose	Banking
`DrishtiModule`	the module's entry point: `start(ModuleContext)`, `stop()`, `health()`; the context gives the kernel services below, its settings, its folders and its logger	`BankingModule`
`ModuleContext` services	`Kinds` (read and subscribe to bound kinds **as the person**), `KindWriter` (write produced kinds to their bound target), `Events`, `Scheduler`, `ComputePools` (incl. the Python pool), `Settings`, `Secrets` (**none**: modules get no secrets; section 9.4), `Audit`, `Registry` (ask, never read), `Caches`, `Clock` (business date)	VaR reads `scenario-pv`, `pnl-vector`, `risk-hierarchy` through `Kinds`

SPI	Purpose	Banking
`KindProvider`	answers a kind the module **computes** (replaces modules' `SourcePlugin`s and the reactive-connector Java runners): fetch, search, subscribe; inputs read through `Kinds`	`position` (derived from trades), `var-result` (`VRES`), `exposure-result`, intraday scenario PVs
`ModuleSettings`	binds the module's `settings/schema.json` to a typed record; validated at start and on change	VaR windows, confidence, root names, hierarchies
`AiToolProvider`	tools the AI assistant may call, each tied to a function (off function = no tool)	"VaR of a scope", "why VaR moved"
`RestModule` (annotation `@ModuleApi`)	marks the module's controllers; the kernel mounts them under the module's namespace (section 4.5)	`/api/v1/risk/**`
`ExportProvider` (P3)	named exports (workbook, regulatory files)	regulatory VaR files
`PlaygroundTemplates` (P3)	the generic workbook streamer's templates	VaR, FRTB-SA, SIMM, SA-CCR, FVA playgrounds
`ModuleMigration` (P4)	upgrade hooks between module versions for its own stored state (no database migrations: state files and caches only)	VaR date caches rebuilt on a format change

The Java SPIs are discovered with `ServiceLoader` **inside the module's class loader** (`META-INF/services/…` in the module's jars) and, for Spring beans, through the module's own child application context (section 4.4).

2.4 What stays kernel-only, and why

Kernel-only	Why
Connector plugins (`SourcePlugin`), storage providers, the lake formats	the site's data, credentials and network belong to the site's administrators; one place to secure and audit them (G4)
Identity, RBAC, the licence check, the registry's model	one trust point (ACCESS_MODEL.md section 2)
The Python runtime, its lock file and sandbox	one runtime (ARCHITECTURE.md section 29.1)
The console shell, menus, Help centre, search, the Elements core	one look, one search, one Help
The database schema files	no migrations, two complete schema files; modules keep state in files under their data folder or in kinds

A new connector type (say, a genomics VCF reader) is a **kernel extension**: a connector plugin jar shipped with the kernel or added to the kernel's `plugins/` folder by the site, signed by the kernel's publisher key; never part of a `.dmod`. A module that needs such a format says so in its data needs (`formats: [vcf]`), and Admin → Data bindings shows *no connector of type vcf is installed*.

2.5 REST namespaces

Prefix	Owner
`/api/v1/{admin,bi,pivot,tables,packs,sutras,views,entities,search,explain,approvals,changes,export,flight,modules,functions,…}`	kernel (the about 100 prefixes of today)
`/api/v1/<api>/…`	a module, `<api>` declared in its manifest (`api.prefix: risk` for banking); unique across installed modules, refused when it clashes with the kernel's list or another module's
`/api/v1/admin/modules`, `/api/v1/admin/bindings`	kernel: module and binding administration (sections 5, 7)
`/m/<module>/static/<version>/…`	console: a module's static files (section 4.7)

Prefix	Owner
`/m/<module>/…` pages	console: a module's pages, or kept at their current addresses when the plugin declares `paths:` (today's `/var`, `/exposure`), refused on clash

2.6 Events

One kernel event bus (in process, gossiped over `drishti-net` where marked). Modules subscribe through `ModuleContext.events()`; a module may publish only events under its own id (`banking.*`).

Event	Payload	Typical module use
`data.loaded`	lake, table, kind, business date, batch id, rows	VaR rebuilds a date when `scenario-pv` or `pnl-vector` lands
`data.changed`	kind, ids or range, cause (correction, restatement)	exposure recomputes a netting set
`binding.changed`	kind, old and new source	VaR drops caches for that kind
`date.rolled`	new business date	hot file prepared
`module.started` / `module.stopped` / `module.refused`	id, version, reason	a dependent module reacts
`settings.changed`	module, keys	VaR reloads hierarchies
`approval.decided`	item, decision	regulatory sign-off
`cache.invalidated`	group	—
`function.switched`	word, on/off	AI tools refreshed

2.7 How the kernel is versioned and shipped

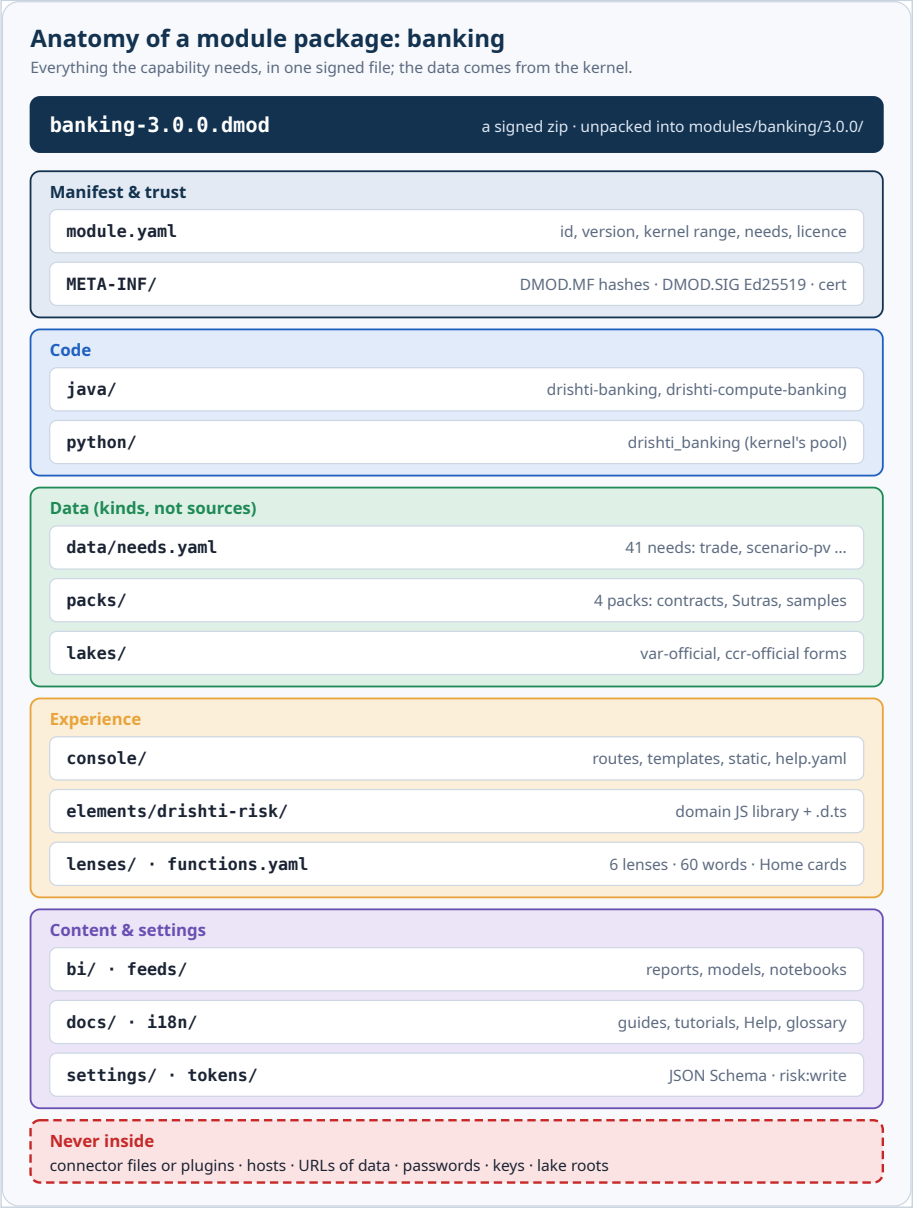
- The kernel ships as `drishti-kernel-<product version>.jar` (the thin jar, P2) with its connector plugins, the console, the Elements core and the kernel's content. Its `kernelApi` and `serves` are in `/public/about`.
- A **distribution build** (section 6.6) is the kernel plus the `.dmod` files the distribution names, in one archive; the modules stay separate files inside it.

3. The module package (`.dmod`)

3.1 File format

- A **zip** (ZIP64 allowed) named `<id>-<version>.dmod`, e.g. `banking-3.0.0.dmod`.
- Signed like a JAR, with a modern signature: `META-INF/DMOD.MF` lists every entry with its SHA-256; `META-INF/DMOD.SIG` is an **Ed25519** detached signature over `DMOD.MF`; `META-INF/publisher.pem` is the publisher's certificate, chained to a key the site trusts (section 9.1). An entry not listed, or listed with another hash, refuses the whole module.
- Uncompressed `module.yaml` first in the archive, so `module info` reads it without unpacking.
- No executable native binaries (`.exe`, `.so`, `.dll`, `.dylib`) at the top level of `python/` or `console/` (Windows QA rule: a module runs on Java and the kernel's Python only); native code a jar bundles (JavaCPP) is allowed and declared (`native: true`), and refused on a site that forbids it.
- Installed **unpacked** into `<modules.dir>/<id>/<version>/` (read-only to the server process), with the original `.dmod` kept beside it for verification at every start.

3.2 Folder layout




```
banking-3.0.0.dmod
├── module.yaml                  the manifest (section 3.3)
├── META-INF/                    DMOD.MF (hashes) · DMOD.SIG (Ed25519) · publisher.pem
├── java/                        the module's jars (own class loader) and their private third-party jars
├── python/                      its Python package(s) for the kernel's pool; requirements.txt (must be within the kernel's lock)
├── data/needs.yaml              the kinds it needs and produces, with contract ranges (section 5)
├── packs/<pack>/                its packs, exactly today's pack folders (pack.yaml, contracts/, sutras/, samples/, python/, guides/)
├── engines/                     engine catalogue entries and engine defaults
├── lakes/                       its published-results lake forms (var-official, ccr-official): tables only, no location
├── bi/                          reports, models, tables, notebooks, fragments, report definitions
├── lenses/                      lenses.yaml (registry lenses: modules, functions, cards, chips) and lens files (menus, Home, Help)
├── functions.yaml               its command words and the APIs behind each
├── console/                     plugin.yaml, routes/, templates/, static/ (drishti_*.js, drishti_*.css), help.yaml
├── elements/<lib>/              its domain Elements library (drishti-risk): ES modules, CSS on tokens, .d.ts, manifest
├── feeds/                       feed definitions and their refresh scripts (.py, with .sh and .ps1 launchers)
├── settings/                    schema.json (JSON Schema 2020-12) and defaults.yaml
├── tokens/token-scopes.yaml     its API token scopes
├── docs/                        guides/, tutorials/, img/, help pages (help.yaml groups), glossary.yaml, README.md
├── i18n/                        message bundles
├── tests/                       its conformance tests (run by `module verify --deep`; never loaded by the server)
└── NOTICES/                     LICENSE, THIRD-PARTY-NOTICES.md
```

The full tree for banking, every folder filled from today's files, is examples/banking/dmod-tree.txt.

3.3 The manifest (`module.yaml`)

The manifest grows from today's `META-INF/drishti/module.yaml` (id, domain, title, feature, requires, contributes); it is still read only by the registry (other kernel parts ask the registry). Every field:

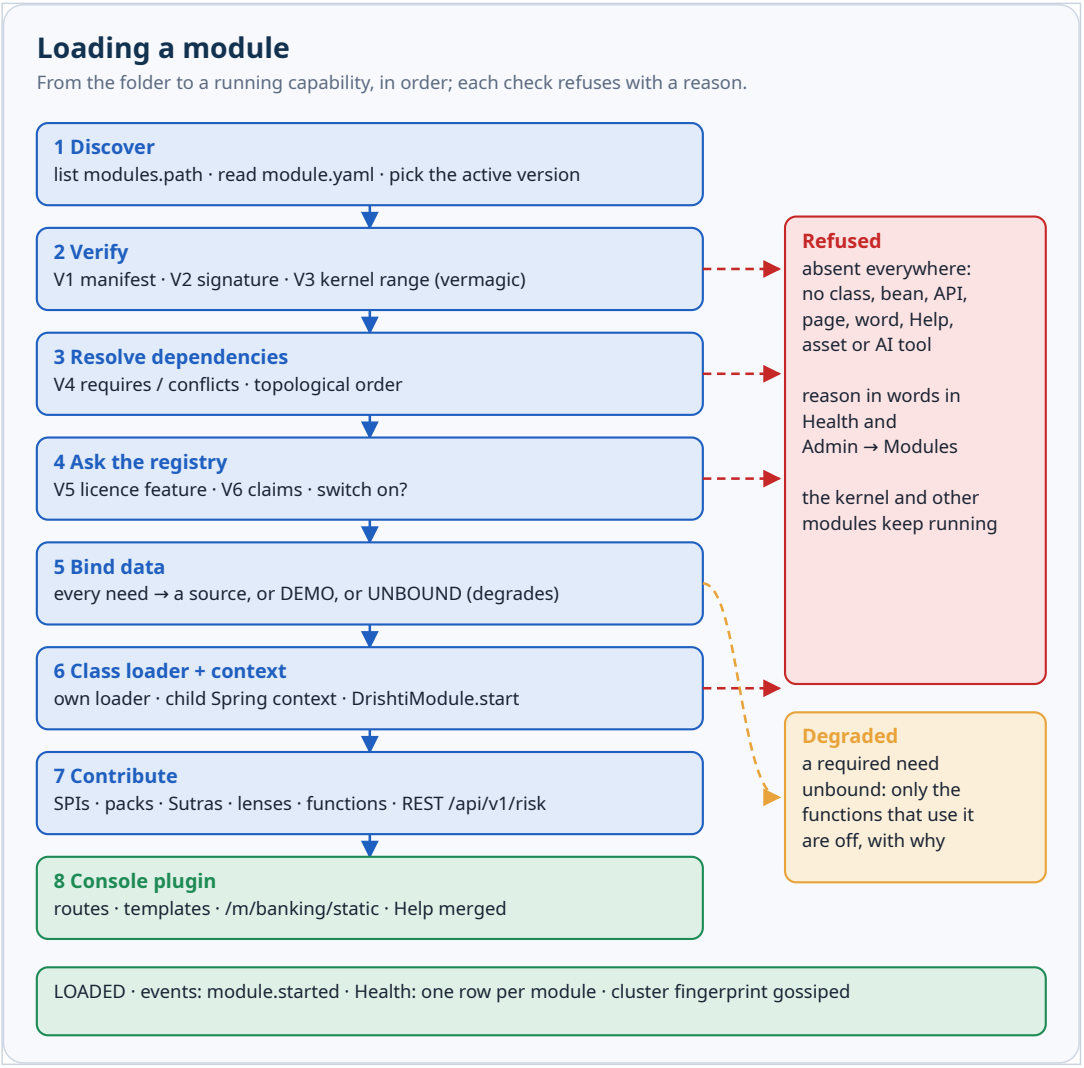
Field	Required	Meaning
`format`	yes	manifest format, `1`
`id`	yes	lower-case id, unique (`banking`); also the folder, the class-loader name, the events prefix
`version`	yes	semver of the module (`3.0.0`)
`title`, `about`	yes	words for Admin, About, Help
`domain`	yes	the domain it serves (`banking`, `economy`, `capitalmarkets`, `genomics`)
`publisher`	yes	`{name, id, contact}`; must match the signing certificate's subject
`licence.feature`	yes	the licence feature that unlocks it (`banking`); `licence.limits` it understands (`positions`, `nodes`)
`kernel`	yes	the kernel API range (`">=1.0 <2.0"`)
`rachana`, `contracts`, `plugin-api`, `elements`	yes	grammar, contract-format, console plugin API and Elements core ranges it was built for
`requires`	no	other modules with ranges, and `optional:` ones it uses when present (`capitalmarkets` optional for banking)
`conflicts`	no	modules it cannot run beside
`provides.kinds`	no	kinds it owns (defined in its packs), by pack
`java`	no	`jars:` (in `java/`), `entry:` the `DrishtiModule` class, `spring:` its auto-configuration, `exports:` packages other modules may *not* see (default none), `native:` true when a jar bundles native code
`compute`	no	its engines (ids, words, pools, switches)
`python`	no	`packages:`, `requirements:`, `pools:` (the kernel pools it submits to, `python` by default), `items:` (Python compute items it registers)
`packs`	no	its packs (folders under `packs/`) and which are on by default
`data`	no	`needs:` file (`data/needs.yaml`, section 5)
`api`	no	`prefix:` (its REST namespace), `aliases:` (old paths answered for one release), `tokens:`
`console`	no	`plugin:` (`console/plugin.yaml`), `paths:` it claims, `elements:` libraries it ships
`functions`	no	`functions.yaml` (or inline): words, their APIs, their module
`lenses`, `home`	no	`lenses/lenses.yaml`; Home cards and chips per lens
`feeds`	no	its feed definitions (each fills one of its kinds)
`bi`	no	its BI folders
`docs`	no	Help sections and pages, guides, tutorials, glossary; `features:` rows for the docs register
`settings`	no	`schema:` and `defaults:`
`events`	no	events it publishes and subscribes to
`resources`	no	expected memory and threads (the capacity planner and the loader's budget check)
`health`	no	its health checks' ids
`taint`	no	set by the kernel, never by the publisher

The full banking example: examples/banking/module.yaml. It lists the two jars (`drishti-banking`, `drishti-compute-banking`), the six engines, the Python package `drishti_banking` with its items, the five packs (with `banking-reference`, open question Q1), the 30-odd data needs, the console plugin and `drishti-risk`, the six lenses, 60 functions, the Home cards, the market-data feed (open question Q6), the seven guides and nine tutorials, the settings schema, the licence feature `banking`, the optional `capitalmarkets` dependency and the kernel range.

Representative stub files with real-looking content:

File	Shows
data/needs.yaml	banking's data needs (section 5.3)
settings/schema.json	the settings schema (VaR windows, confidence, root names, hierarchies)
packs/market-risk/sutras/var-desk.v1.sutra.yaml	one Sutra as the module ships it
docs/help/var-in-five-minutes.md	a Help page stub
console/plugin.yaml	the console plugin as packaged (today's file, paths made module-relative)

4. Loading



4.1 Module paths

Setting	Default	Meaning
`drishti.modules.path` (`DRISHTI_MODULES_PATH`)	`${drishti.data.dir}/modules`	an ordered list of folders holding installed modules (`<id>/<version>/` unpacked + the `.dmod`); first match of an id wins; read-only to the server
`drishti.modules.inbox`	`${drishti.data.dir}/modules-inbox`	where `module install` and Admin → Modules put an uploaded `.dmod` waiting for approval
`drishti.modules.active`	`config/modules/active.yaml`	which version of each installed module is active (written by install, upgrade, rollback; four eyes)
`drishti.modules.trust`	`config/trust/publishers/`	publisher certificates the site trusts
`drishti.modules.<id>.enabled`	`true`	the existing switch (ACCESS_MODEL.md section 8)

All folders live under the configurable data folder (no root litter, visible names); the server never writes to a module's folder.

4.2 Discovery

At start, before any bean: list every `<path>/<id>/<version>/module.yaml`; pick the version `active.yaml` names (else the highest); parse each manifest against the manifest schema; build the dependency graph. Nothing else of a module is touched yet.

4.3 Verification, in order

Each step refuses with a reason code and words; a refused module is **absent everywhere** (section 4.9).

#	Check	Refusal (example words)
V1	manifest valid (schema, `format: 1`, id = folder)	*banking 3.0.0: module.yaml line 12: `kernel` missing*
V2	signature: `DMOD.MF` hashes match the unpacked files and the `.dmod`; `DMOD.SIG` verifies; certificate chains to a trusted publisher; publisher id = manifest's	*signature does not verify (file java/drishti-banking-3.0.0.jar changed)*
V3	kernel range (`vermagic`): `kernel`, `rachana`, `contracts`, `plugin-api`, `elements` ranges contain the kernel's	*built for kernel API >=2.0; this kernel is 1.4*

#	Check	Refusal (example words)
V4	dependencies: every `requires` present, in range, and itself not refused; no cycle; no `conflicts`	*needs capitalmarkets >=1.2; 1.1 installed*
V5	licence through the registry: the licence names the feature, is valid on this date, within limits	*the licence does not include banking*
V6	claims: its API prefix, paths, functions, kinds and lens ids clash with no kernel or other module's	*function VAR is claimed by banking and riskx*
V7	data needs: every `provides.kinds` contract parses; every `needs` contract range is satisfiable by some module or kernel kind	a missing *binding* does not refuse: it degrades (section 5.6)
V8	resources: its declared memory fits the node's budget	*needs 6 GB; 3 GB free in the module budget* (warning, or refusal when `drishti.modules.strict-resources`)
V9	enabled: the site switch	not a refusal: *off by the administrator*

Order: topological by `requires` (a module after everything it requires), then by id for a stable order. A refusal propagates to every module that requires it (*refused: requires banking, which is refused*).

4.4 Java: one class loader per module



- **Shared (parent-first)**: `com.ash.drishti.kernel.api..`, the Java platform, Spring Framework and Boot, Jackson, SLF4J, Apache Arrow, Micrometer, Jakarta Servlet and Validation. The list is part of the kernel API (a MINOR may add, only a MAJOR removes).
- **Private (child-first)**: every other class in the module's `java/` jars, including its own copies of libraries (a module may ship commons-math 3.6 while the kernel uses another version).
- **Hidden**: every kernel class outside the API (`com.ash.drishti.server..` etc.) is not visible to a module (`ClassNotFoundException`); an architecture test in the SDK fails the build earlier.
- **Spring**: each module gets a **child** `ApplicationContext` with the module's class loader; its parent exposes only the kernel API beans (the `ModuleContext` services). Its `@ConditionalOnModule` stays as a guard. SPI implementations are beans or `ServiceLoader` entries in that context; the kernel collects them through the registry of contributions (`ModuleContributions`), never by scanning a module's packages.
- **No module sees another module's classes**; modules talk through kinds, functions and events.
- **Stop**: `DrishtiModule.stop()`, then the child context closes, the contributions are withdrawn, the class loader is dropped (P5 hot unload depends on this being leak-free; a leak test loads and unloads 50 times in the SDK harness).

4.5 REST

The module's controllers (`@ModuleApi`) are mounted by the kernel under `/api/v1/<prefix>/`, behind the kernel's filters in this order: TLS/identity → module gate (`DRS-1025` when off) → function gate (`DRS-1026`) → permission (`DRS-1027`) → audit → the module's `DispatcherServlet` (one per module, bound to its child context). A module cannot register a filter, a servlet or a path outside its prefix. `ApiAlias` keeps working for one release.

4.6 Python compute

- The module's `python/` packages are made importable **only in that module's runs**: a run submitted by `banking` gets `<module>/python` prepended to its sandbox's import path; another module's runs never see it. The kernel's `drishti_kernel` package and the locked libraries are shared.
- **No pip at install time.** `python/requirements.txt` must be satisfied by the kernel's lock file; a module needing a library outside it is refused with *needs scipy-stats-x 1.2, not in this site's Python lock: an administrator approves it first* (ARCHITECTURE.md section 29.6).
- Every run goes through the kernel's one door (`PythonRuns`): same strict sandbox, limits, audit line, the person's row rules on every frame. The audit line gains `module`.
- A module's Python items (BI python visuals, Python measures, its compute packs) are registered by the manifest's `python.items`, shown in Build with the module's name.

4.7 The console plugin

- **Where**: the console reads the same `modules.path` (or its own `modules.path` pointing at the same folders: one cluster setting). For each module the server reports as *loaded* (`/public/about`), the console mounts `<module>/console/`.
- **Python code**: imported as `drishti_modules.<id>` (a namespace package rooted at the module folder), not `plugins.<id>`; the `plugin.yaml` format stays (`routers`, `registers`, `code`, `engines`, `words`, `paths`, `templates`, `assets`, `screens`, `help`).
- **Templates**: the module's `console/templates` folder is added to the template loader after the kernel's; a module template may extend kernel layouts (`terminal/_base.html`); a kernel template never includes a module's (rule 1).
- **Static files**: served from `/m/<id>/static/<version>/…` with a year-long cache (the version busts it), only to people for whom the module is on and their lens uses it (today's `_module_asset_refused`, unchanged). The `drishti_` prefix stays.
- **Elements**: the module's `elements/<lib>/` is added to the Elements manifest with `module: <id>`; the loader's existing refusal applies.
- **Help merge**: `console/help.yaml` heads and pages (today's `help-sections:` / `sections:`) merge into Help; the guides and tutorials are read from the module's `docs/` folder; the docs register gains the module's `docs.features` rows, so the docs gates check a module's docs like the kernel's.
- **Sitemap and menus**: module menu entries carry `requires: module:<id>` as today; lens files from `lenses/` replace `config/lenses/<module>/`.

4.8 Order of start

1. Kernel core (settings, registry, licence, identity).
2. Discovery and verification of modules (4.2, 4.3).
3. **Data bindings resolved** for every need (section 5): before any module starts, so a module starts knowing which of its needs are bound.
4. Modules started in dependency order: class loader, child context, `DrishtiModule.start`, contributions registered.
5. Packs loaded (kernel packs, then modules' packs in module order), Sutras, lenses, functions.
6. Kernel services open (REST, console, scheduler, feeds); `module.started` events.

A module's start has a time budget (`drishti.modules.start-seconds`, 120); exceeded → refused (*did not start in 120 s*), the kernel goes on.

4.9 Failure: refused means absent

A refused module (any of V1–V8, a start failure or a crash in `start`) is treated exactly as *not installed*: no class loaded (its loader is dropped), no bean, engine, job, API, pack, connector binding, route, menu, search hit, command, Help page, asset, AI tool, export or embed. What remains:

- **Health**: one row per module: `LOADED`, `LOADED (degraded: 2 data needs unbound)`, `OFF (administrator)`, `REFUSED (reason)`, `TAINTED`.
- **Admin → Modules**: the same with the full reason, the check that failed, and what to do.
- **Words for people**: a typed function of a refused module answers *VAR is not available: the banking module is not running (signature does not verify). Ask your administrator.* (administrators see the reason; others see *not available*).
- **Kernel keeps running**; a refused module never stops the kernel. A refused *required-by-distribution* module (section 6.2, `required: true`) makes Health `DEGRADED` for the whole instance.

4.10 Restart versus hot load

What	P2	P5
Packs, Sutras, lenses, Help, BI content inside an installed module	**hot** (the pack reloader, as today)	hot
Settings of a module	hot where the schema marks a key `hot: true`; else *restart needed*	same
Data bindings	hot (section 5.8)	hot
A module's code (Java, Python package, console Python)	**restart**: install/upgrade/remove stages the change, Admin shows *restart needed*, a rolling restart applies it	**hot**: unload (stop, close context, drop loader), load the new version; refused when a leak check fails
A module's static assets	with the code (versioned path)	same

The development build: the SDK's harness and the developer's IDE run the kernel with a module folder pointing at the module's build output (`target/dmod/`), same loader, same checks, a development key (section 8.3). There is no "on the class path" second path.

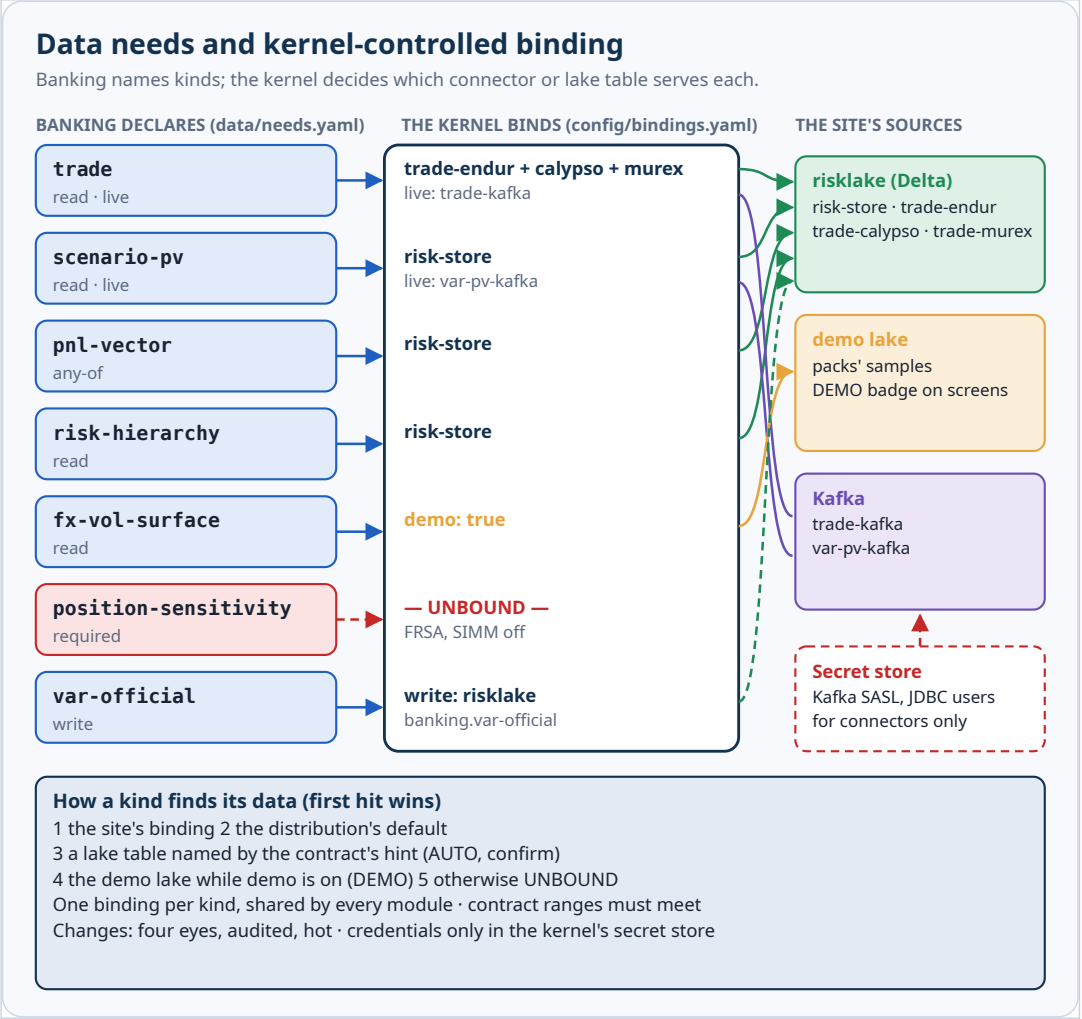
4.11 Cluster consistency

- Every node must run the **same set of module ids and versions** and the same licence (today's licence rule, extended). Each node gossips its *module fingerprint* (sorted `id@version#sha256-of-DMOD.MF`) over `drishti-net`.
- A node whose fingerprint differs from the cluster's **coordinator** (the owner node) does not join the serving set: Health on every node shows *node-3 runs banking 3.0.0, the cluster 3.0.1*; requests are not routed to it.
- Install and upgrade are **cluster operations**: the approved `.dmod` is copied to each node's inbox (or read from shared EFS paths, which are identical on AWS), then a rolling restart node by node (P2); a node starts only if its fingerprint matches the target.
- Bindings, active versions and switches are cluster configuration (the shared overlays already used for connectors and load overrides).

---

5. Data needs and connector binding

The heart of requirement 4: **a module says what data it needs; the kernel decides where that data comes from.**



5.1 Who owns what

Thing	Owner	Where
Connector **types** (plugins)	kernel	`plugins/drishti-plugin-*`, the kernel's plugin folder
Connector **configurations** (a Kafka cluster, a JDBC database, a Delta root on S3)	the site's data administrator	`config/connectors/<name>.yaml` (unchanged), credentials as references `${secret:…}`
Credentials	the site's security administrator	the kernel's secret store (environment, a secrets file outside `config/`, or an external vault)
Lakes and their tables	the site	`config/lakes/<lake>.yaml` (unchanged)
Kinds and their contracts	the module that defines them (`provides.kinds`), or the kernel (core kinds)	the module's packs (`packs/<pack>/contracts/*.contract.yaml`)
Data needs (which kinds a module reads or writes, which contract versions, required or optional)	the module	`data/needs.yaml` in the `.dmod`
Bindings (kind → which connectors / lake tables serve it, in what read order, live or not, and where produced kinds are written)	the site's data administrator, **through the kernel**	`config/bindings.yaml` (new) + Admin → Data bindings, four eyes
Default bindings for a product	the distribution	`bindings.defaults` in the distribution file (section 6)

5.2 Contracts: the format the kernel owns

The contract format is today's `*.contract.yaml` (kind, mnemonic, pack, version, class, grain, key, dated-by, cadence, fields with type and presence, vectors) with **one change**: the `storage:` block (which today names `connector: risk-store`, `plugin: delta`, `table:`) is **removed from the contract** and replaced by a connector-free **hint**:

```
binding-hint:                # what a site usually binds; never a connector name, plugin or location
  lake-table: risk.scenario-pv          # the table name a lake file would use
  partition: [BUSINESS_DATE]            # the first partition is always the business date
  live: optional                        # a live source makes sense (intraday)
  demo: samples/scenario-pv.jsonl       # the demo lake's sample for this kind
```

The kernel reads `contract-format: 1`; a module declaring `contracts: ">=1 <2"` is checked at V3.

5.3 Data needs (`data/needs.yaml`)

Each need: the kind, the contract version range, `use` (`read`, `write`, `live`), `required` (true / false), `any-of` groups, the functions and engines that depend on it, and its `demo` allowance. Banking's needs, abridged (the full file: examples/banking/data/needs.yaml):

Kind	Contract	Use	Required	Used by	Notes
`trade`	`>=1.0 <2`	read, live	yes	TRD, VAR (scope by trade), EXP, PREDEAL	a union of trade stores (Endur, Calypso, Murex)
`scenario-pv`	`>=1.1 <2`	read, live	any-of A	VAR, PVAR, SMAT	per trade, own currency; intraday stream
`pnl-vector`	`>=1.0 <2`	read	any-of A	VAR, PVAR, SMAT	instead of scenario PVs
`fx-scenario`	`>=1.0 <2`	read	with scenario-pv	VAR	converts scenario PVs to USD
`scenario-set`	`>=1.0 <2`	read	yes	VAR, SWIN	the 509-day and 250-day windows
`risk-hierarchy`	`>=1.0 <2`	read	yes	VAR, VLIM, HIER	the VaR tree and limit letters
`portfolio`	`>=1.0 <2`	read	no	VAR PORT	
`var-limit`	`>=1.0 <2`	read	no	VLIM, limit bars	
`desk-pnl`	`>=1.0 <2`	read	no	backtesting	actual and hypothetical P&L
`pnl-history`	`>=1.0 <2`	read	no	SWIN	stress window selection
`position-sensitivity`	`>=1.0 <2`	read	yes for FRSA, SIMM	FRSA, SIMM, SENS	
`risk-factor`	`>=1.0 <2`	read	yes for FRSA, SIMM	RFAC	
`netting-set`, `csa`, `agreement`, `collateral-balance`	`>=1.0 <2`	read	yes for EXP	EXP, PREDEAL, SIMM	
`credit-limit`	`>=1.0 <2`	read	no	limit use	computed use, stored limits
`ir-curve`, `fx-spot`, `fx-vol-surface`, … (market data)	`>=1.0 <2`	read	yes for EXP; no for VaR	EXP paths, TVAL	from capital markets when installed (Q6)
`counterparty`, `legal-entity`, `book`, `desk`, `trader`	`>=1.0 <2`	read	yes	every screen	`banking-reference` (Q1)
`calendar`	kernel `>=1.0`	read	yes	business dates	kernel kind
`var-official`, `ccr-official`	own	**write**	no	VOFF, XOFC, official results	written to a lake the site binds
`position`, `var-result`, `exposure-result`	own	**computed**	—	POS, VRES	`KindProvider`s; not bindable

5.4 Bindings (`config/bindings.yaml`)

Bindings are per kind, not per module. A kind is one dataset at a site; every module that needs it reads the same binding. The file (examples/site/bindings.yaml) has three example sites in one:

```
bindings:
  trade:
    read: [trade-endur, trade-calypso, trade-murex]   # connectors (config/connectors/<name>.yaml), in read order
    union: true                                        # one trade store across them
    live: trade-kafka                                  # live updates (a Kafka connector)
  scenario-pv:
    read: [risk-store]                                 # a Delta connector on risklake (lake: risklake)
    live: var-pv-kafka
  risk-hierarchy:
    read: [risk-store]
  var-official:
    write: { lake: risklake, table: banking.var-official }
  pnl-vector:
    demo: true                                         # explicitly the demo lake (a test site)
```

Key	Meaning
`read`	connectors that serve the kind, in read order (today's `readOrder`: Redis for recent days, Delta behind it)
`union`	rows from all of them (today's `union: true` on the trade connectors)
`live`	a connector that pushes changes (Kafka, RabbitMQ, ActiveMQ); subscribers get them through `Kinds.subscribe`
`write`	where a produced kind is written: a lake and table (the kernel's writer; the module never names a path)
`demo`	`true` binds the demo lake's samples explicitly
`for:` (exception)	a per-module override, `for: {genomics-qc: [qc-store]}`; allowed only for kinds marked `shareable: per-module` in their contract (Q4)

Today's equivalent is spread out: a connector file's `kinds:` list and its `lake:`; the contract's `storage.connector`. P1 folds both into `bindings.yaml` (no dual machinery): connector files keep `plugin`, settings and `lake:`, lose `kinds:`; `drishti.py bindings migrate` writes `bindings.yaml` from today's files once, and a connector file still carrying `kinds:` is read for one release with a deprecation line in Health.

5.5 How a kind finds its data (resolution)

For each kind any loaded module needs, in this order, first hit wins:

1. **The site's binding** in `config/bindings.yaml` (as approved).
2. **The distribution's default** (`bindings.defaults`, section 6), e.g. `drishti.finance` binds banking's kinds to `risklake` tables by their hints when a lake file holds them.
3. **The lake convention**: a configured lake (not demo) whose lake file has the contract's `binding-hint.lake-table` table, read by a connector of the lake's format → bound, marked `AUTO` (Admin shows *found by convention: confirm*).
4. **The demo lake**, only while `drishti.demo.enabled` and the need allows demo: marked `DEMO` (amber everywhere: "demo data").
5. Otherwise **UNBOUND**.

Demo and real never mix (DATA_LAKE_GUIDE.md): a kind is either demo or real; a pack whose kinds are bound to more than one lake is `DEGRADED` (today's warning, moved from packs to bindings).

5.6 What a missing or bad binding does

State	Meaning	Effect
`BOUND`	a source serves it; its schema satisfies the contract range	normal
`AUTO`	found by convention	normal; Admin asks to confirm
`DEMO`	demo samples	works; "DEMO DATA" badge on screens using it
`STALE`	bound, but no load within the contract's cadence	works; Health amber *scenario-pv last loaded 2026-11-04 (cadence daily)*
`MISMATCH`	the source's columns or types break the contract (a required field missing)	**treated as unbound for that need**; Health red with the columns
`UNBOUND`, required	nothing serves it	the module **loads but is degraded**: functions that depend on the need are off with the reason (*VAR needs scenario-pv or pnl-vector: no source is bound; ask your data administrator*); others work
`UNBOUND`, optional	nothing serves it	info only; dependent features hidden (e.g. backtesting without `desk-pnl`)

`any-of` groups are satisfied by one bound member. Health lists `needs[]` per module; `GET /api/v1/admin/bindings` answers the same.

5.7 Computed kinds and the end of module connectors

Today banking ships connector plugins (`var-positions`, `var-pv-stream`) and reactive connectors (`var-engine`, `exposure-engine`). Under G4 these are not connectors: they **compute** a kind from other kinds. They become `KindProvider`s in the module: `position` and `var-result` are *computed kinds* (`provides.kinds[].computed: true`), never bindable, always the module's. The transport part of `var-pv-stream` (reading Kafka) is the kernel's Kafka connector bound as `scenario-pv`'s `live:` source; banking subscribes to the kind. Connector files written for the old names keep loading for one release (today's `moved-classes.properties` mechanism).

5.8 Changing a binding

Admin → Data bindings (mock-up b) or `drishti.py bindings set scenario-pv --read risk-store`: proposed, approved by a second administrator (four eyes), audited, applied **hot**: the kernel validates the new source against the contract, swaps the route, publishes `binding.changed`; modules drop their caches for that kind. A binding change never needs a module restart.

5.9 Two modules, one kind

- One binding per kind: banking and a hypothetical `collateral` module both read the one `trade` binding.
- **Contract ranges must intersect**: if banking needs `trade >=1.0 <2` and `collateral` needs `>=1.3 <2`, the bound source must satisfy 1.3+; V7 refuses a module whose range cannot meet the kind owner's version (*collateral needs trade >=2.0; banking provides 1.4*).
- **One owner per kind**: the module that defines it in its packs; another module that needs it `requires` the owner (optionally) or uses a kernel kind. Two modules defining the same kind id → V6 refuses the second.
- Per-module views of a kind are row rules and masks (permissions), not separate bindings; the `for:` exception is Q4.

5.10 Credentials never inside a module

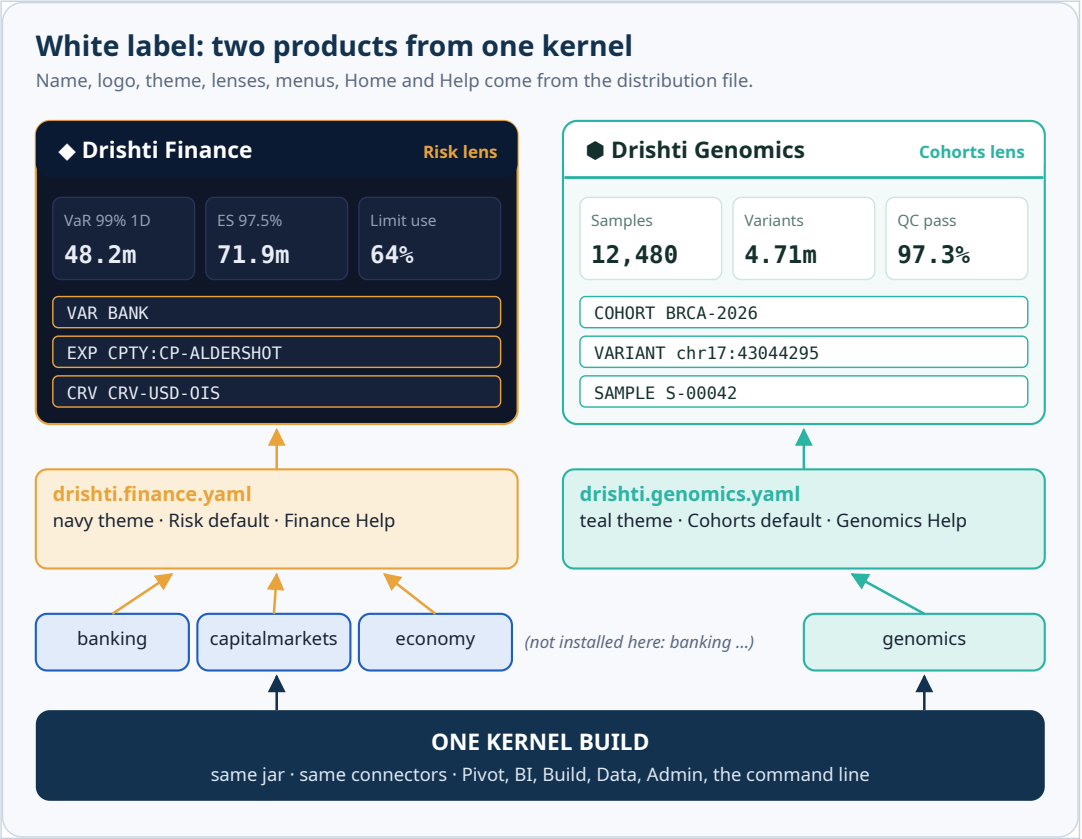
- A module's files contain no connector, plugin, host, URL of a data source, user name, password, key or token: the packager and V1 scan for connector keys (`plugin:`, `connector:`, `bootstrap-servers`, `jdbc:`, `password`, `secret`, `token`) and known secret patterns, and refuse.
- At run time a module has no API to read connector settings or secrets; `Kinds` returns rows, never the connection.
- Feeds a module ships (public sources such as the ECB rates) name a public URL and need no credentials; one that needs a key reads it from the kernel's secret store by a **name the site provides** in the feed's binding (`feeds.market-data.api-key: ${secret:ecb}`), not from the module.

5.11 Banking's example bindings, three sites

Site	`trade`	`scenario-pv` / `pnl-vector`	`risk-hierarchy`	`var-official`
Laptop demo	`DEMO`	`DEMO`	`DEMO`	written to the demo lake
Bank on Delta (`risklake`)	`trade-endur` + `trade-calypso` + `trade-murex` (union) on risklake	`risk-store` (Delta, risklake)	`risk-store`	`risklake` table `banking.var-official`
Bank with live trades	the same + `live: trade-kafka` (Kafka, SASL credentials from the secret store)	`risk-store` + `live: var-pv-kafka`	`risk-store`	the same

6. The distribution file (white label)

6.1 What it is



One YAML file, `config/distribution.yaml` (or `DRISHTI_DISTRIBUTION=<path>`), that turns the kernel into a product: which modules, which brand and theme, which lenses and Home, which docs, which default bindings. It folds today's scattered product settings (`drishti.edition`, `ui.product`, `ui.copyright`, `drishti.branding.*`, the lens files' `title`, `config/packs/default-packs.yaml`) into one place, **as the one way**: the old keys are read for one release when no distribution file exists.

6.2 Fields

Field	Meaning
`distribution`	id (`drishti.finance`)
`version`	the distribution's own version
`kernel`	the kernel product version range it was tested with
`brand`	`product` (name in the top bar), `product_native`, `tagline`, `owner`, `copyright`, `logo` (light/dark files), `favicon`, `about` page text, `support` contact
`theme`	the theme token set (`finance-navy`), overrides of tokens (`--ac`, `--bg`...), default mode (dark/light/system), density
`modules`	each module id with `version` range, `required` (Health degraded if refused) and `enabled` default
`lenses`	which lenses are offered, the default lens, per-lens title overrides
`home`	the default landing per lens, cards and chips order, the welcome text
`menus`	top-level menu order; menu entries hidden for the whole product
`features`	presets of switches (today's `config/lenses/features.yaml`)
`docs`	which Help sections and tutorials show, their order, the docs title ("Drishti Finance Help"), extra pages from the distribution's folder
`bindings.defaults`	default bindings for this product (section 5.5)
`demo`	whether the demo lake ships and is on
`glossary`	extra terms
`locale`	default language and formats

6.3 Examples

- examples/distributions/drishti.finance.yaml: banking + capital markets
- economy; navy theme; Risk lens default; Help in the banking order; risklake defaults.
- examples/distributions/drishti.genomics.yaml: a hypothetical `genomics` module + the kernel; teal theme; Cohorts lens; Help with the kernel's generic guides and the genomics module's; no banking word anywhere.

6.4 How Help and docs follow

Help is assembled at start from the kernel's pages plus each loaded module's pages, filtered by the distribution's `docs` (sections and order), then by the person's lens and functions (as today). The product name in every page is the brand's (`{product}`), never a literal (the branding rule: the 26-literal fix becomes a P3 gate). Exported PDFs, the About page and the console title use the brand. Docs built for print (`docs build --distribution drishti.finance`) render the same selection.

6.5 What a distribution may not do

It cannot add code, connectors or credentials; it names modules, never contains them. It cannot widen a licence (the licence still decides). It cannot rename functions (command words are the modules').

6.6 Distribution builds

`drishti.py dist build drishti.finance.yaml` (P3) makes `drishti-finance-<version>.zip`: the kernel jar, the console, the connector plugins, the named `.dmod` files (unchanged, signed by their publishers), the distribution file and the brand's assets. Installing it is unzipping it; the modules are installed into `modules.path` on first start (approved by the installer's act, audited).

---

7. Lifecycle and tooling

7.1 The command line

Command	Does
`drishti.py module new <id> --domain <d> [--template content|compute|full]`	a new module source tree from the SDK template (section 8.2)
`drishti.py module build [--sign-with dev|<key>]`	builds the jars (Maven), runs the module's tests against the SDK harness, checks (no secrets, no kernel internals, docs gates, contracts), writes `target/dmod/<id>-<version>.dmod`
`drishti.py module sign <file> --key <publisher key>`	signs (or re-signs) a `.dmod` with a publisher key (an HSM or a key file; never stored in the repo)
`drishti.py module verify <file> [--deep]`	V1–V7 against a running or offline kernel; `--deep` runs its conformance tests
`drishti.py module install <file> [--activate]`	uploads to the inbox and proposes the install (four eyes); with approval: unpacks, writes `active.yaml`, *restart needed*
`drishti.py module list [--json]`	`lsmod`: id, version, state, licence, taint, needs bound/unbound
`drishti.py module info <id|file>`	`modinfo`: the manifest, signature, publisher, contributions, needs
`drishti.py module remove <id> [--version v]`	proposes removal (four eyes); refused while another active module requires it
`drishti.py module upgrade <file>` / `rollback <id>`	install a newer version beside the old; switch `active.yaml`; rollback switches back (both versions stay until `prune`)
`drishti.py bindings list | set | migrate | check`	section 5
`drishti.py dist build | check <file>`	section 6

Transcripts: module new (docs/architecture/kernel-and-modules/examples/cli/module-new.txt), module build (docs/architecture/kernel-and-modules/examples/cli/module-build.txt), module install (docs/architecture/kernel-and-modules/examples/cli/module-install.txt), module list (docs/architecture/kernel-and-modules/examples/cli/module-list.txt).

7.2 Admin → Modules

The screen of mock-up (a): installed modules with version, publisher, state, licence, needs, taint; refused ones with the reason and the fix; *Install module...* (upload `.dmod` → verify report → propose); *Remove*, *Upgrade*, *Roll back*, *Switch off*; a pending-approvals strip (four eyes); *restart needed* banner with the nodes affected.

7.3 Four-eyes install

Install, upgrade, rollback, remove and every binding change are **proposals** in the Approvals inbox (the kernel's one approvals engine): the proposer cannot approve; the approver sees the verify report (signature, publisher, kernel range, dependencies, licence, needs and which would be unbound, resource budget, what it contributes: functions, APIs, lenses, Help) and the effect (*adds 60 functions, 6 lenses; needs 3 bindings you have not made*). Approved → applied at the next (rolling) restart in P2.

7.4 Upgrade and rollback

- Versions are installed side by side (`banking/3.0.0`, `banking/3.0.1`); `active.yaml` picks one.
- Upgrade: verify the new one with the current set (dependents' ranges still met), approve, restart (rolling), `module.started` with the new version; the module's `ModuleMigration` (P4) upgrades its own state files.
- Rollback: switch `active.yaml` back, restart; state written by the newer version must be readable by the older or the module declares `rollback: false` for that pair (then rollback is refused with words).
- `module prune` deletes inactive versions after a retention (default: keep one previous).

7.5 Health and taint

Taint (Linux's word, the owner's frame) marks an instance that runs something unsupported; it does not stop anything:

Taint	Cause
`D` development-signed	a module signed with the SDK's development key (allowed only when `drishti.modules.allow-dev: true`)

Taint	Cause
`O` override	an administrator forced a module past a soft check (resource budget)
`R` out-of-range	`drishti.modules.ignore-kernel-range: true` (development only)
`U` unknown publisher	a publisher added to trust by the site, not Drishti's chain

Health shows *TAINTED (D: banking-dev 3.1.0-SNAPSHOT)*; support bundles carry it.

---

8. The Module SDK

8.1 What it contains

Part	Contents
`drishti-kernel-api-<v>.jar`	the SPIs and service interfaces of section 2.3, Javadoc; Maven coordinates `com.ash.drishti:drishti-kernel-api` (internal repository)
`drishti-kernel-testkit-<v>.jar`	the harness (8.3)
`@drishti/elements-*` `.d.ts`	the Elements core typings (from ELEMENTS.md's distribution channels), the console plugin hooks' typings
`drishti_kernel` Python	the compute SDK's types and a local sandbox runner for tests
schemas	`module.schema.json`, `needs.schema.json`, `contract.schema.json`, `plugin.schema.json`, `distribution.schema.json` (editors validate as you type)
templates	`module new` templates: `content` (packs, Sutras, lenses, docs only: like economy), `compute` (+ Java compute and Python), `full` (+ console plugin and a domain Elements library)
packager	`module build` / `sign` / `verify` (Python tool + Maven plugin `drishti-dmod-maven-plugin`)
a sample	`genomics-sample` module (content + small Java compute), used by the SDK's own tests and by `drishti.genomics`

8.2 The template (`module new genomics --template full`)

```
genomics/
├── module.yaml
├── pom.xml                          parent: drishti-module-parent (kernel API as `provided`)
├── java/src/main/java/…/GenomicsModule.java      implements DrishtiModule
├── java/src/test/java/…/GenomicsModuleTest.java  boots the harness with this module
├── python/drishti_genomics/__init__.py
├── data/needs.yaml
├── packs/genomics/{pack.yaml,contracts/,sutras/,samples/}
├── console/{plugin.yaml,routes/,templates/,static/}
├── elements/drishti-genomics/{index.js,drishti-genomics.css,index.d.ts,manifest.json}
├── lenses/lenses.yaml
├── settings/{schema.json,defaults.yaml}
└── docs/{README.md,guides/,tutorials/,help.yaml,glossary.yaml}
```

8.3 The test harness (kernel + one module)

- `KernelHarness.with(module("target/dmod")).demoLake().start()` boots the **real kernel** (thin jar, in process) with only this module (and its `requires`, fetched from the internal repository), a temporary data folder (`@TempDir`), the demo lake from the module's samples, a development licence and key (taint `D`).
- Gives: a REST client as a test person, the `Kinds` API with fixture bindings, a console client, a Python pool, the module-off variant (proves "gone everywhere" for this module automatically), and the **conformance suite** every module must pass: manifest valid, no kernel internals, no secrets, Elements only (no bespoke UI guards), docs gates, its functions answer, off means gone, session isolation (two people at once), class loader unloads cleanly.
- Python side: `pytest` plugin `drishti_kernel.testing` (the console harness with the module mounted).

8.4 The developer guide (outline)

A new guide, `docs/guides/MODULE_DEVELOPER_GUIDE.md` (P2; one guide + one tutorial, per the docs rule): 1. What a module is; the kernel API and its versions. 2. `module new`, the layout. 3. Data: kinds, contracts, needs; testing with the demo lake; never connectors. 4. Java: `DrishtiModule`, the SPIs one by one with examples, REST under your prefix. 5. Python compute items. 6. Console: plugin.yaml, pages from Elements, your domain library, CSS on tokens. 7. Lenses, functions, Home cards. 8. Help, guides, tutorials, glossary. 9. Settings schema. 10. Build, sign, verify, install. 11. Tests and the conformance suite. 12. Upgrades, deprecations, compatibility. 13. Security rules. 14. Worked example: banking. Tutorial: *Build the genomics sample module from scratch*.

---

9. Security

9.1 Signing

- Every `.dmod` is signed (Ed25519 over the hash list). Drishti's **module publisher root** signs publisher certificates (Drishti itself for banking, economy, capital markets; partners later). The site trusts the root by default (`config/trust/publishers/drishti-root.pem`) and may add its own publishers (taint `U`).
- Keys: private keys in an HSM or an offline key file, never in a repository; `module sign` takes a PKCS#11 URI or a file.
- Revocation: a revocation list file shipped with each kernel release and loadable by the site; a revoked certificate refuses its modules.

- The kernel jar and connector plugins are signed the same way (P4).

9.2 The licence

The signed licence of ACCESS_MODEL.md section 7 (built in i, after the security discussion) names **features**; each module's `licence.feature` must be in it (V5). Limits (`positions`, `nodes`, `users`) are read by the registry and offered to the module through `ModuleContext.licence()` (read-only, the module cannot change them). The licence check is the kernel's; modules know nothing of it beyond the limits.

9.3 Sandboxing

Part	Isolation
Java	own class loader, sees only the kernel API; no `System.exit`, no new threads outside the kernel's pools (an SDK architecture test; the kernel's thread factory names threads `mod-<id>-…` for attribution); file access only to its own data folder through `ModuleContext.folders()` (checked by the conformance suite; the JVM has no Security Manager any more, so this is a review and test rule, not a runtime wall: Q5)
Python	the kernel's strict sandbox (no network interface, Landlock, audit hook), per-run import path, the person's row rules on every frame
Console Python	runs in the console process; restricted to its plugin hooks; the static import guard (no core imports of modules, no module imports of other modules)
JavaScript and CSS	served from `/m/<id>/static/<version>/`, under the console's CSP (`script-src 'self'` plus SRI hashes from the module's manifest; no inline script, no `eval`, no remote origins)

9.4 Secrets

Never in a module (5.10). A module has no secret API. Its own sensitive settings (none for banking) would be references the site fills (`${secret:name}`) and only the kernel resolves.

9.5 Session isolation

A module runs in a multi-user, concurrent server: the module conformance suite runs two people at once and fails on any per-person state in statics, singletons or shared caches without the person's identity (the session-isolation rule; cross-person leaks are P1 and release blockers). `Kinds` reads are always **as the person**.

9.6 Audit

Install, upgrade, rollback, remove, switch, binding change, refusal, taint, and every run of a module's API and Python items carry the module id and version in the audit line.

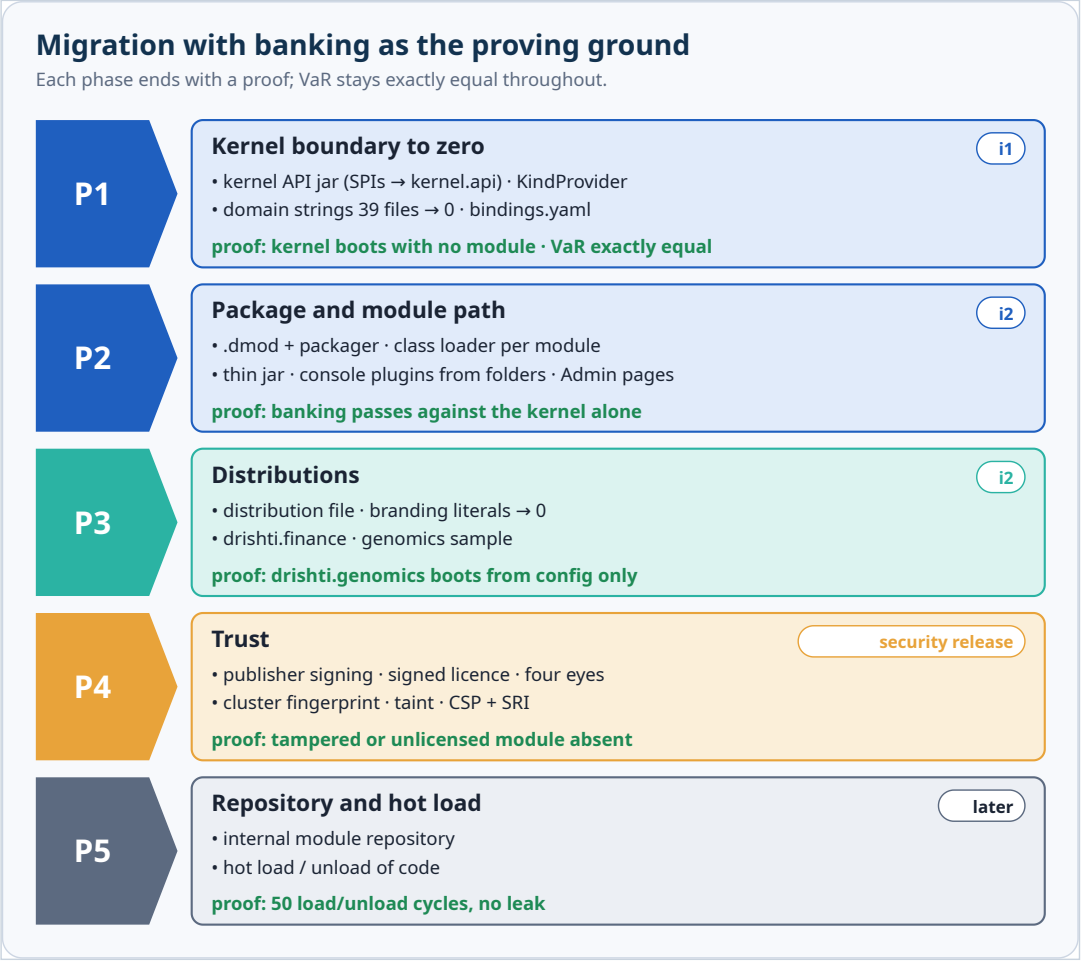
---

10. Migration, with banking as the proving ground

10.1 Where we are (surveyed on `89f010df5`)

Already modular (built in the e+f+g train)	Still leaking or not done
Java split: `drishti-banking`, `drishti-compute-banking`; cycles J1–J6 broken through SPIs; `ArchitectureRulesTest`	**39 kernel files with 77 banking strings** on the rule-4 allow list (`tools/domain_strings_allowed.yaml`): bicompute (Explain, functions, what-if, scale), the console's `core/edition.py` (14), `core/facts.py`, `drishti_pivot.js`, `EngineCatalogue`, `ModuleManifest`, `EmbeddedAccessRegistry`, `ErrorCode`
Manifests (`META-INF/drishti/module.yaml`, `lenses.yaml`, `engines.yaml`, `token-scopes.yaml`) in the jar; the registry decides	**The thin jar is not built**: banking is always on the class path; "installed" = in the fat jar
`@ConditionalOnModule`, module switch, `DRS-1025` / `1026`; module off = gone everywhere (tests)	**Console plugins live inside the repo's console** (`drishti-console/plugins/{banking,economy}`), imported as `plugins.<id>`
Console plugin hooks C1–C7; static assets by module and lens; Elements loader refuses a module's library	**Modules depend on `drishti-server` internals** (no kernel API jar yet)
Packs, lakes, BI, tables, reports, lenses in module folders (`config/*/banking/`)	**Banking ships connector plugins** (`var-positions`, `var-pv-stream`) and reactive connectors; contracts name connectors (`storage.connector`)
Editions folded into lenses: done (`config/lenses/<module>/`)	**No signing, no module path, no distribution file**; branding split across `ui.*`, `drishti.branding.*` and lens titles
Packs name their domain; Help by module (`plugins/banking/help.yaml`)	Banking Python CLIs (`varcli.py`, …) and the guides/tutorials live in the repo's shared folders

10.2 Phases



Phase	Deliverables	Proof (gate)
P1 · Kernel boundary to zero	`drishti-kernel-api` jar (section 2.3: existing SPIs moved under `kernel.api`, new `DrishtiModule`, `ModuleContext`, `KindProvider`, `ModuleSettings`, `AiToolProvider`, `@ModuleApi`); banking compiled against it only; the 39-file allow list to **0**; `KindProvider` replaces banking's connector plugins; contracts' `storage:` → `binding-hint:`; `config/bindings.yaml` + `bindings migrate`; kernel API 1.0 frozen with `japicmp` baseline	kernel boots with **no modules** (economy and capital markets off too) and serves BI, Pivot, Build, Help, GLEIF; `domain_strings` empty; banking passes its suite; **VaR exactly equal**
P2 · The package and the path	the `.dmod` format and packager (`module build`, unsigned dev key), `modules.path`, discovery and V1–V4, V6–V7 checks, class loader per module + child Spring context, REST mounting, Python per-module import path, console plugins loaded from module folders (`drishti_modules.<id>`, `/m/<id>/static/`), Help merge, **the thin jar**, `module list | info | install | remove | verify`, Admin → Modules (mock-up a), Admin → Data bindings (mock-up b), launchers/Docker/ECS/stack with the modules folder; the SDK harness; banking, economy and capital markets as `.dmod`s; the developer guide + tutorial	**banking passes against the kernel alone** (the harness: kernel thin jar + `banking.dmod`, nothing else); the full drill on the thin jar + three modules; a drill with **no modules**; VaR exactly equal
P3 · Distributions	the distribution file and `dist build`; branding literals to 0 (the 26-literal fix and its guard); `drishti.finance` (banking + capital markets + economy) as the default product; the **`genomics-sample`** module and `drishti.genomics`; white-label mock-up (c) built; Help/docs by distribution	**`drishti.genomics` boots from configuration only** (same kernel build, a different file and module set): no banking word on any page, in search, Help or assets (a crawler test); `drishti.finance` equals today's product (screenshots)
P4 · Trust (with the security release)	signing with publisher keys, trust store, revocation, V5 with the signed licence, four-eyes install/upgrade/rollback, cluster fingerprint and rolling install, taint, `ModuleMigration`, CSP with SRI for module assets, signed kernel and connector plugins	tampered `.dmod` refused; unlicensed module absent; mixed-version node kept out of the serving set; rollback restores VaR exactly
P5 · Repository and hot load (later)	an internal module repository (search, fetch, dependency download), hot code load/unload with the leak test, module marketplace only if the owner wants one	50 load/unload cycles without a class-loader leak; hot upgrade under load with no failed request

P1–P3 are i's plug-and-play track (the owner to decide whether to split i: its size is 2–3× g); P4 with the security release; P5 later.

10.3 Banking step by step (P1–P2)

1. Survey again; freeze the boundary list (refresh DOMAIN_MODULES.md section 2).
2. Create `drishti-kernel-api`; move the SPI interfaces (package rename `com.ash.drishti.kernel.api.*`; the old types deleted, not aliased: banking is the only implementer and moves in the same branch).
3. Banking's `pom.xml` depends on `drishti-kernel-api` (`provided`) and drops `drishti-server`; fix each compile error by a kernel API service or a new SPI; the architecture test *module imports only the kernel API* turns on.

4. Empty the allow list file by file: words to the module's manifest, engine knowledge to the engine catalogue, `edition.py`'s banking words to lens files, `drishti_pivot.js`'s MR bits to the plugin's `screens.pivot`.
5. `var-positions`, `var-pv-stream` → `KindProvider`; `var-engine`, `exposure-engine` reactive connectors → computed kinds; contracts' `storage:` → `binding-hint:`; `bindings migrate` on the demo and test sites.
6. The packager: lay banking's files into the `.dmod` layout (a `git mv` of the plugin, packs, lenses, BI, lakes, docs into a `modules/banking/` source tree in the repo; the repo remains one repository, the module builds separately from it).
7. The thin jar; launchers gain the modules folder; the drill installs `banking.dmod`.
8. The VaR check, both drills.

10.4 Risks and mitigations

Risk	Mitigation
VaR drifts during the moves	the exact-equality VaR check on every banking branch's pre-merge gate, not only at the drill
Spring child contexts break auto-configuration ordering or AOP across loaders	prototype in week 1 on `economy` (content-only) then on banking's smallest engine (SIMM); keep the shared library list small; a test per SPI crossing the boundary
Class-loader leaks (threads, caches, ThreadLocals)	kernel-owned thread pools only; the leak test in the harness from P2 even though hot unload is P5
Size of i (2–3× g)	split: i1 = data standards + P1; i2 = P2 + P3 (owner Q8)
Packaging change across every launcher, Docker, ECS, stack and the drill (why the thin jar was deferred)	one branch for all launchers with a launch-matrix test; the drill's first run on the thin jar is a dry run without timing
Two ways during migration (in-jar and `.dmod`)	none: the branch that introduces the module path removes the in-jar path in the same merge; tests and harness use the `.dmod`
Python code paths across modules	per-run import path; a test that one module's run cannot import another's package
Console import renames (`plugins.banking` → `drishti_modules.banking`) break bookmarks or extensions	addresses kept via `paths:`; only Python import paths change (internal)
Performance (an extra class-loader hop, per-module DispatcherServlet)	measured in the benchmark: VaR request latency and pivot throughput before/after, budget ≤ 2%
Windows QA (no .exe)	the packager and loader are Java + Python only; `.dmod` is a zip; signature checks in Java; tests run on Windows
Docs churn (guides move into modules)	guides keep their slugs; Help merges them; the docs gates read module docs

11. Open questions for the owner

#	Question	Recommendation
Q1	The core pack's finance entities (`book`, `desk`, `trader`, `counterparty`, `legal-entity`, ...): kernel or banking? (was D1)	move them to a **`banking-reference` pack in the banking module**; the kernel keeps `calendar` and generic entities (organisation, person, location); GLEIF stays kernel
Q2	Bindings file vs connectors' `kinds:` lists	**`config/bindings.yaml` is the one home**; `kinds:` read for one release, then removed
Q3	Module static files at `/m/<id>/static/<version>/` (new addresses) or today's `/static/…`	**new addresses** (versioned cache, per-module CSP, clean unload); page addresses unchanged
Q4	Per-module bindings of one kind (`for:`)	**not in i**: one binding per kind; per-person views are row rules; revisit with a real case
Q5	Java isolation without a Security Manager: accept review + tests, or run untrusted modules out of process	**in process for Drishti-signed and partner modules**; out-of-process modules (a gRPC/Flight boundary) only if a third-party marketplace is wanted (P5+)
Q6	The market-data pack and feed: banking or capital markets	**capital markets owns market data** (pack and feed); banking `requires: capitalmarkets` (optional today, required once its engines read it); closes DOMAIN_MODULES D6's interim
Q7	Who signs: one Drishti key for everything, or publisher certificates under a Drishti root	**a root + publisher certificates** (Drishti, then partners), revocation list per release
Q8	Split i	**yes**: i1 = data standards + P1 (+ performance); i2 = P2 + P3; P4 with the security release
Q9	Repository layout: banking's sources stay in this repository under `modules/banking/` or move to their own repository	**stay in this repository** (one drill, one history) but build separately into a `.dmod`; a separate repository when a partner module exists
Q10	Default distribution of the repo's build	**`drishti.finance`** (today's product); `drishti.genomics` as a sample only, never shipped to customers
Q11	The kernel API version's start	**1.0 at the end of P2** (P1's API is 0.x and may change freely)

12. What moves from DOMAIN_MODULES.md

DOMAIN_MODULES.md remains the record of the split as built (sections 1–5, 8) until P2 completes, then is deleted (git keeps it) and its rules move into the developer guide. From today:

DOMAIN_MODULES.md	Moves to
section 6 step 9 (the thin jar and the plugins folder) and section 8 *Phase 2, still to do*	this document, section 10 (P2)
section 7 D1 (core pack's finance entities), D6's interim (market-data pack)	section 11, Q1 and Q6
section 4 rule 4 (strings) with its allow list	section 1.3 principle 1 and P1's gate (empty)
section 3 (the layout as built)	stays until P2, then replaced by section 3.2 here

ACCESS_MODEL.md stays the home of the access chain and the registry; this document adds the package, the loader and binding, and only links back.

---

13. Mock-ups

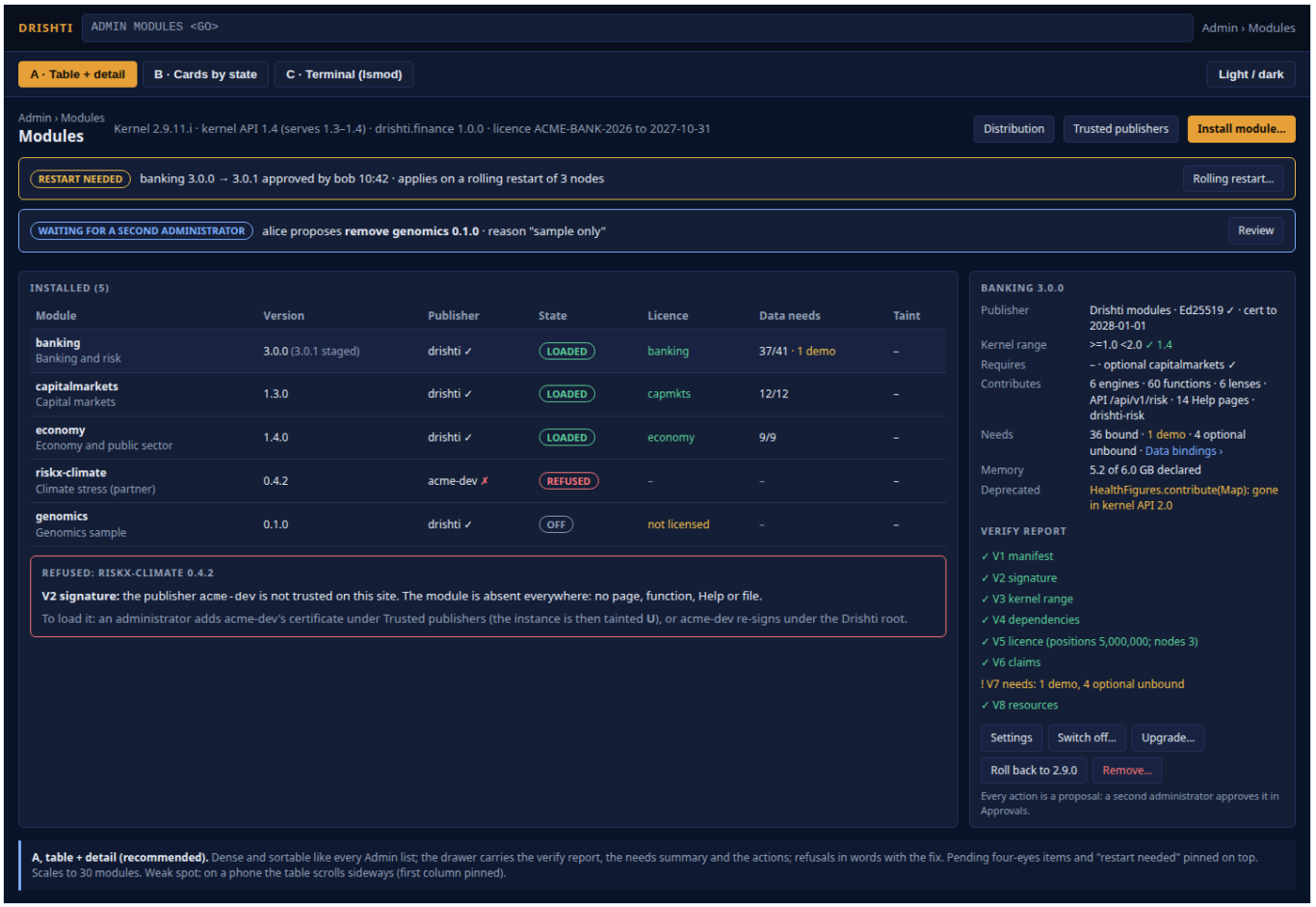
Static, self-contained (no CDN), in Drishti's terminal look with light and dark; each file holds the alternatives as views of one page (A the recommended default, per the A/B/C rule), with the trade-offs written under each.

Screen	File	Alternatives	Recommendation
(a) Admin → Modules	mockups/admin-modules.html	A table + detail drawer · B cards by state · C terminal `lsmod`	**A** (dense, sortable, the drawer carries the verify report and four eyes); C as the terminal layout
(b) Admin → Data bindings	mockups/data-bindings.html	A needs matrix (kind × module, source per row) · B by module (needs list per module) · C data map (sources → kinds → modules)	**A** (one row per kind shows sharing and gaps at once); B as the module's own tab; C later as a read-only picture
(c) White label: Finance vs Genomics	mockups/white-label.html	A side by side · B one console with a distribution switch · C the distribution file beside its result	**A** for the owner's review; C is the authoring view in Admin → Distribution (P3)

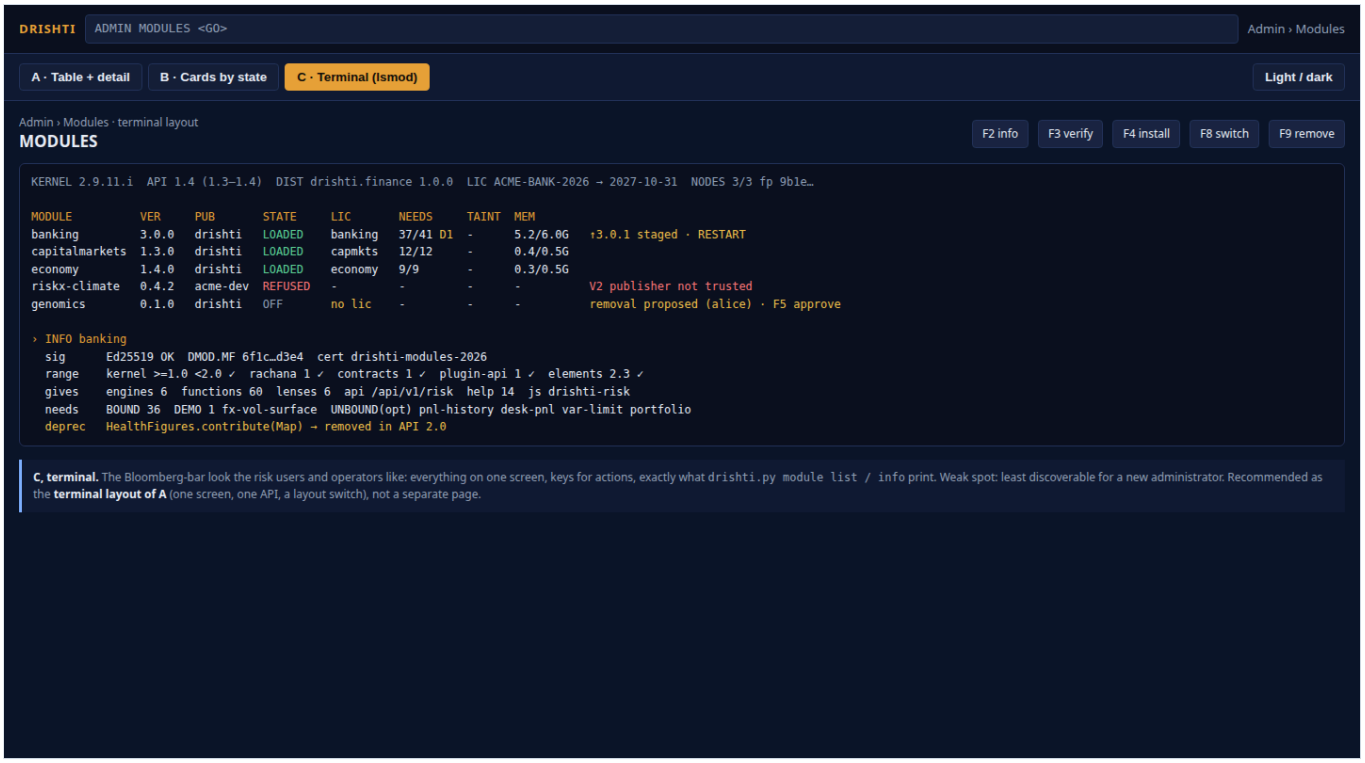
13.1 The recommended options

Screenshots of the static pages (dark theme; each page also has light). The other options are in the same pages (buttons A, B, C) and as pictures beside them (`admin-modules-B.png` …).

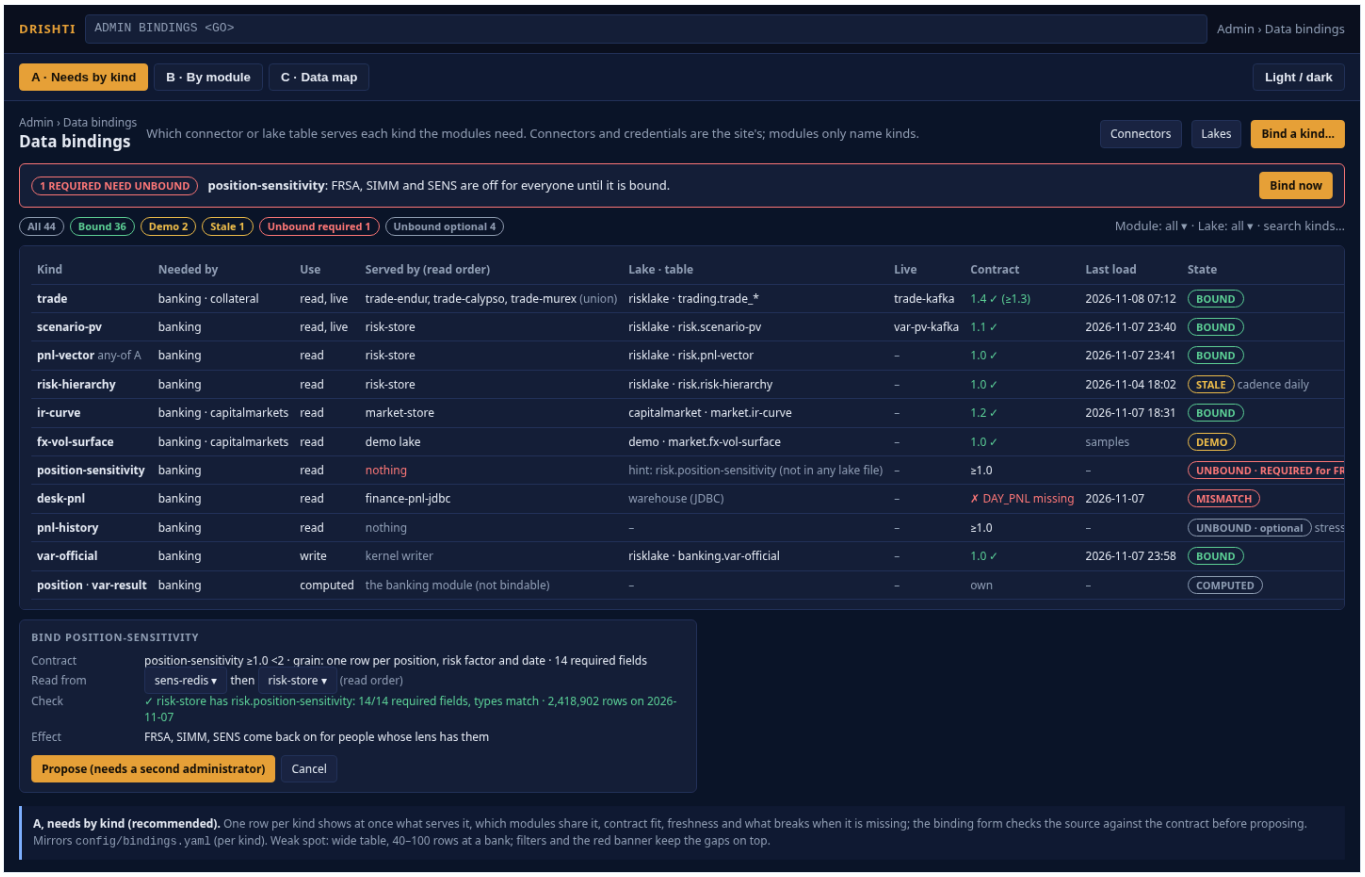
(a) Admin → Modules, option A (table + detail drawer, the verify report, four-eyes items and *restart needed* on top):



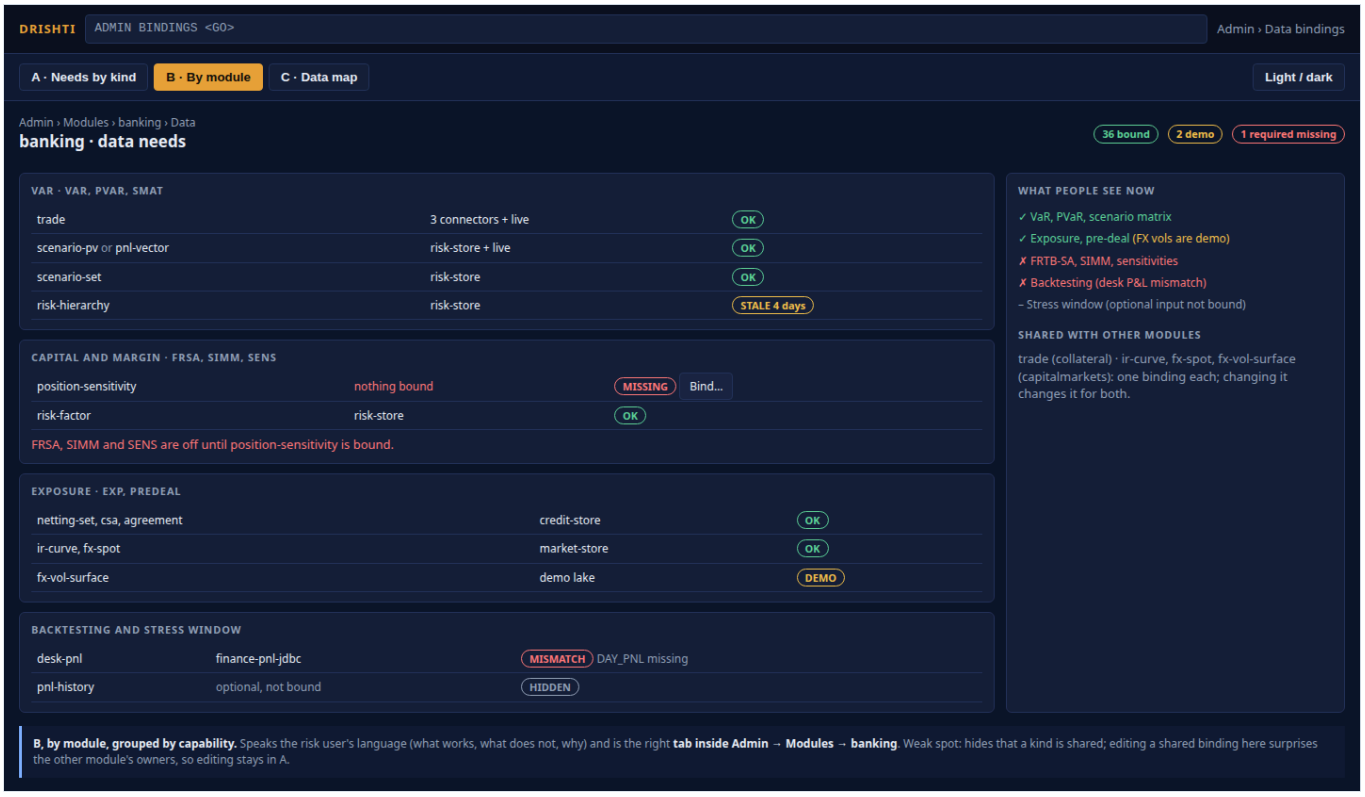
(a) Admin → Modules, option C (the terminal layout of the same page):



(b) Admin → Data bindings, option A (needs by kind, sharing, contract fit, freshness; the bind form checks before proposing):



(b) option B (by module, the tab inside Admin → Modules → banking):



(c) White label, option A (Drishti Finance and Drishti Genomics side by side, one kernel):

